the hidden state. For each decoder layer l, we train the corresponding prefix $c_{\text{opt}}^{(l)} = [c_1^{(l)}, ..., c_M^{(l)}]$ to maximize the probability of the model yielding the exact subsequent phrase after the original context. In particular, we conduct the same causal intervention in the hidden states h_T^l. We then optimize the probability distribution of the subsequent generation under the learned context to be the same as the original model when all its previous generation is given correctly:

$$\text{argmin}\, \text{KL}(\hat{y}_{M+N}\, ;\, y_{T+N}) \tag{10}$$

Where the predicted distribution $\hat{y}_n$ is given using the same intervention as described in Eq. 9:

$$\hat{y}_{M+n} = G([c_1, ..., c_M, x_{T+1}, .., x_{T+N}] \\ ||\, h_M^l := h_T^l) \tag{11}$$

We hence optimize this objective with the model frozen and only prefix left to be trained. Notably, our approach is different from the implementation of prefix tuning (Li and Liang, 2021) in the sense that we back-propagate the gradient through the model instead of a temporary MLP, as empirically it produces a significantly better optimized context.

3 Experiments and Results

3.1 Data

We perform evaluation on samples of the Pile (Gao et al., 2020), which is the 825GB dataset used to train GPT-J-6B (Wang and Komatsuzaki, 2021) as well as other LLMs.

To train the linear models, we sample 100,000 tokens that have an average of 518 sized-context. Amongst the 100,000 token samples, we use 10,000 of them to train for our learned prompt experiment. For testing our methods, we sample another 1000 tokens that have an average previous context length of 535. To simplify our analysis of the degree to which single hidden token representations encode subsequent n-grams, we draw our samples from contexts in which the original transformer model made a correct prediction.

More specifically, we randomly sampled train and test data points from the subset of token locations where the autoregressive transformer under consideration correctly predicts the following token. In Table 1, we break down the types of tokens present in the testing data by categorizing the last token (T) of the prefix as well as the generated tokens outputs of GPT-J , through greedy (argmax) decoding, at $N = 0, 1, 2, 3$ with respect to various properties, such as whether they are lower-cased tokens that start with a space, or are numerical tokens, and so on.

3.2 Evaluation Metrics

For evaluation we adopt the same metrics used in prior related work Din et al. (2023), namely Precision@k and Surprisal.

Precision@k measures the appearance of the top probability token in the output at N tokens ahead we predict from the hidden state with respect to the observed top-k tokens from GPT-J-6B model output. Higher values are better here because these mean the actual token at the corresponding future token was accurately predicted.

Surprisal, on the other hand, is the minus log probability according to the GPT-J-6B model output of the highest probability token according to the proposed probing methods. Lower is better for this measure because such values imply that the top predicted tokens are deemed probable by the model.

3.3 Experimental Setup

Linear Model We train two types of linear models — one with an output space of 4096 (the hidden representation size used by GPT-J-6B), and the other one with 50,400 (the vocabulary space of the same). GPT-J-6B comprises 28 layers. We train 4 instances for each of these layers, one for each different "future" token position we consider ($n = 0, 1, 2, 3$). As input we accept the source hidden state, i.e., h_T^l. Our output is either the hidden state, i.e., h_{T+N}^L or the decoded output at the position (vocabulary distribution) $T + N$.

Fixed Prompt Causal Intervention This is an evaluation-only setup where we choose four generic context prompts and perform causal intervention on these contexts as shown in Figure 2. The four fixed context prompts that we test are:

- `Hello! Could you please tell me more about "`
- `The multi-tokens present here are "`
- `The concepts in this hidden state listed are: (`
- `<|endoftext|> This state is describing about the following concept:`

The hidden states are gathered from layer l of the last token of the context tokens and are transplanted into the hidden representation of the last token in the generic prompts at the same layer l.

Properties	Last Original Context Token	N = 0	N = 1	N = 2	N = 3	Examples
Lowercase No Space	12	14.5	18.1	13.1	13.4	'itability', 'aka', 'ension'
Lowercase With Space	42	39.1	37.1	38.4	36.7	' sense', ' tests', ' punitive'
Uppercase No Space	2.4	2.7	2.2	2.8	1.6	'V', 'TABLE', 'SE'
Uppercase With Space	1.9	2.4	1.1	1.5	1.7	' STAR', ' UK', ' USA'
Token length < 4	57.8	59.8	64.3	59.9	63.2	'*', 'ate', '</'
Token length ≥ 4	42.2	40.2	35.9	40.5	37	' validation', ' Subaru', 'ulsion'
Punctuation	15.7	14.5	17.3	15.2	19	'-', '.', '</'
Numerical	2.4	2.7	1.9	3.2	2.8	'1998', '001', '5'

Table 1: Data Frequency of different token properties on the Last Prefix Tokens and GPT outputs at N=0,1,2,3. Each number in the table is a percentage of the test dataset, which is of size 1000.

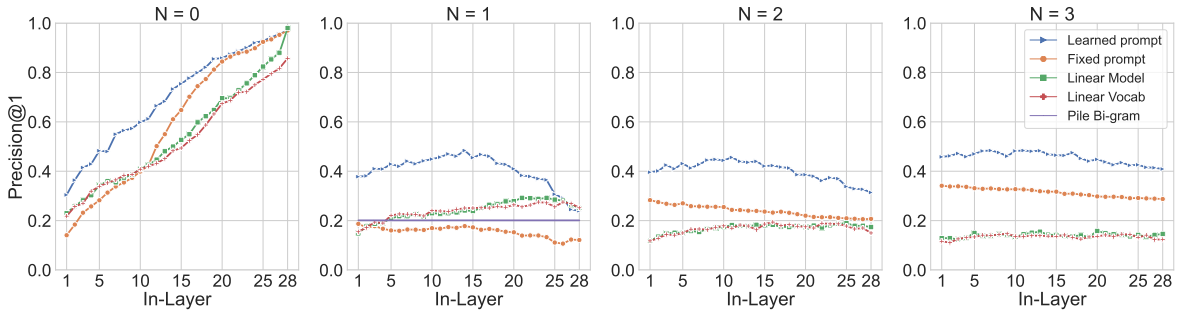


Figure 4: Accuracy (Precision@1) using the transplanted hidden representation. The $N = 0$ case models immediate next-token prediction, and $N \geq 1$ are the subsequent-token cases that are the focus of our work. The learned prompt is best able to recover future token information from hidden states of a preceding individual token, with predictive accuracy peaking at middle layers, with more than double the accuracy of a bigram baseline. A linear model predicting the hidden state fares comparably to predicting directly into the output vocabulary.

Learned Prompt Causal Intervention We then compare with trained prompts with the same token length as the fixed prompts. We train a soft prompt for each layer l from 1 to 28. Each learned prompt is trained by maximizing the probability of generating the token from the prefix context at the penultimate layer, when the hidden state is transplanted at layer l at the last token of the soft prompt, in the same way as the fixed prompts are applied. We train a prefix with a length of 10. This method performs best and is our main method.

3.4 Unveiling Subsequent Tokens

Figure 4 and Figure 5 illustrate the difference between our method and the baselines. The learned prompt optimized with the objective of predicting the next token (N=1) has the best performance. On average, the precision@1 is 24.8% higher, precision@5 is 25.3% higher, and precision@10 is 25.1% higher than the **best** baseline method. The bigram baseline[1] at N=1 is shown as a horizontal line; the bigram model i achieves 20.1% accuracy. For surprisal, the learned prompt also has the lowest value, which indicates its efficacy at maximally unveiling the information behind the hidden states.

3.5 Contexts of Accurate Predictions

To further explore the contexts in which these methods seem better (or worse) able to predict subsequent tokens, we categorize input token (the last original context token) into eight (non-mutually exclusive) categories, shown in Table 3. We report the model accuracies when using layer 14, where the learned prompt model peaks.

While all categories of token types are predicted better by the learned prompt than by the linear model, the relative improvement is highest when the last context token is a lowercase token preceded by a space, or a longer token. This suggests that information about how to complete long words may not be immediately accessible by a linear model decoder, but that they can be made accessible by using the parameters of the pretrained model as done by the learned prompt intervention method.

[1]The bigram baseline is collected from 900,000 documents from the Pile dataset.

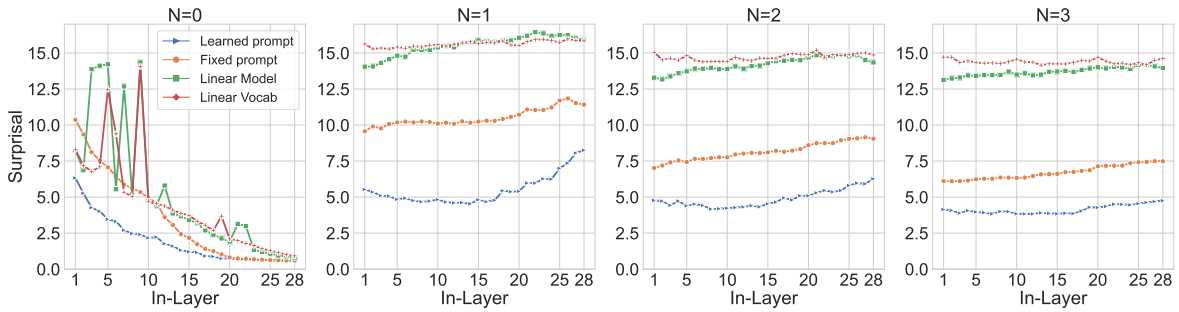


Figure 5: Average surprisal of the model after transplantation. Again the learned prompt performs best, confirming the presence of subsequent-token information encoded at middle-layer hidden states.

	LENS	N=1	N=2	N=3
Accuracy				
LEARNED	97.0	**48.4**	**43.7**	**46.9**
FIXED	97.0	20.8	30.0	36.5
HS	**98.0**	29.2	19.0	15.8
VOCAB	85.7	27.5	19.4	14.7
Surprisal				
LEARNED	**0.6**	**4.5**	**4.4**	**3.9**
FIXED	**0.6**	8.8	6.5	5.7
HS	0.8	14.1	13.2	13.1
VOCAB	0.9	15.3	14.4	14.2

Table 2: Best accuracy and surprisal results for each method. LEARNED refers to the Learned Prompt Causal Intervention Method; FIXED denotes the Fixed version. HS is the Linear Model variation that predicts Hidden State; VOCAB, is the Linear Model variation that predicts a distribution over the vocabulary directly.

We have also observed that the accuracy of predicting subsequent tokens is correlates with the model's confidence in its next token prediction. In the case of $N = 1$, for instance, the learned prompt intervention method's calibrated accuracy is 26%, 57%, 77%, and 95% for model confidence groups of 0-30%, 30-60%, 60-90%, and 90%-100%, respectively. These trends appear in $N = 2$ and $N = 3$ as well. This suggests that we might gainfully use this decoding method as a probing tool, trusting that predicted future tokens are generally accurate when the model is confident.

Does future information appear only in the presence of higher-level concepts? For example, one might hypothesize that in cases the language model predicts an entire named entity, that the probing method might decode future predictions more accurately. To investigate this, we performed sub-group analyses on test results to characterize how well the best probing method performed specifically for multi-token named entities. Interestingly, we found little difference: when examining just the named entity cases, we observe similar or slightly lower accuracy: 44%, 42% and 37% for $N = 1, 2, 3$, suggesting that future information is present broadly, not only for long entity names.

In sum, we have found that a single hidden state encodes information about outputs more than one token ahead, and we have demonstrated three different methods that can decode them for GPT-J-6B.

Application: Future Lens We apply Learned Prompt Intervention to create a novel probing tool we call the *Future Lens*. Given a soft prompt, we perform the intervention using the states arising from the user's prompt to provide a view into what the hidden states encode about future tokens. In Figure 6, we show an example for the prompt: "Marty McFly from". The Future lens reports the anticipated four tokens from every hidden state in the model (across layers).

In the Future Lens visualization, every cell represents a hidden state from a particular layer ("L{digit}") at a specific token. The shade of each cell indicates the model's average confidence with respect to the corresponding token predictions (darker shades indicate greater confidence). For example, at the cell representing the hidden state at Layer 25 at the token "from", we can see that the confidence in the predicted tokens "Back to the Future" is strong. This particular state suggests that the LLM already knows that Marty McFly is related to the Back to the Future movie. Interestingly, the model also assumes "Marty" to have the surname Donough. Returning to the predictions at token "from", we see that the early layers seem to first predict locations such as Australia or Boston. However, through future predictions, we can see the model begins to associate Marty McFly with a

Last Context Token Type	Linear: Vocab Space	Linear: Hidden State	Fixed Context	Learned Context
Lowercase No Space	21.7	25.2	9.2	**32.5**
Lowercase With Space	26.4	20.8	19.2	**51.9**
Uppercase No Space	**29.2**	26.3	0.0	**23.3**
Uppercase With Space	26.3	26.3	10.5	**31.6**
Token length < 4	26.5	24.9	21.8	**46.9**
Token length ≥ 4	23.9	24.4	18.0	**52.1**
Punctuation	28.7	28.7	16.6	**47.8**
Numerical	12.5	16.7	20.8	**33.3**

Table 3: Accuracy of predicting $N = 1$ token ahead (y_{T+1}, which predicts x_{T+2}) based on hidden representation of the last context token(x_T). Results are shown for layer $l = 14$, where the learned prompt model is most accurate.

movie around Layer 6. Hence, through this tool, we can gain further insights about the model's chain of predictions at every hidden state.

4 Related Work

Knowledge Prediction and Manipulation Recent works have delved into LLM internals to better understand how such models predict the next token at each computation step. Geva *et al.* (2021), for instance, find that the feed-forward layers in transformers operate as key-value memories, allowing one to intervene at those layers to modify the next token output (Geva et al., 2022). Frameworks such as ROME (Meng et al., 2022a) and MEMIT (Meng et al., 2022b) scale such manipulations to edit knowledge in stored in LLMs.

The consensus that has emerged in these papers is that some early-middle and late layer calculations contribute the most to the final predicted token. Tools such as Logit lens (nostalgebraist, 2020) and Tuned lens (Belrose et al., 2023; Din et al., 2023) allow us to look at the top-k values of the transformer at *every* layer and token to see early next-token predictions. Katz and Belinkov (2023) used logit lens to visualize semantic information flow in GPT-2 models. While these works primarily deal with next-token predictions, Hernandez et al. (2023) shows that specific attributes of an entity can be extracted with an affine transformation on the entity representation long before the LM is actually required to predict the attribute, enabling an "attribute lens" on early layers and early tokens. We aim to characterize how the current hidden state would affect the prediction of both the next token and tokens farther ahead, but unlike Hernandez et al. (2023), we deal with open contexts and are not constrained to certain relations.

Early Exit Decoding To optimize the running time and space requirements of training models, prior work has looked at "early exit" strategies, which usually involves stopping at earlier layers of computation and estimating the final predictions based on those computations (Schuster et al., 2022; Xin et al., 2021; Kong et al., 2022; Zhang and He, 2020; Din et al., 2023). The takeaway from these methods is that it is possible to achieve prediction performance comparable to that observed when all layers are used even when dropping a couple of computational layers for each token. For instance, Din and colleagues (2023) used linear transformations to predict a later layer's hidden representation from an earlier layer at the same token. This approach was able to preserve $\sim$95% of the full transformer model outputs on GPT-2 (Radford et al., 2019) and BERT (Devlin et al., 2018). This result implies that initial model layers encode information that to largely determines the final output. In this work we test the limits of this phenomenon by evaluating the degree to which a single hidden state for a token at position T can be used to predict tokens multiple steps ahead (i.e., at $T + N$).

Memorization in Language Models Due to the potentially sensitive information present in the datasets used to train language models (LMs), past work has investigated what, when, and why memorization occurs (Carlini et al., 2021, 2019; Feldman and Zhang, 2020; Lehman et al., 2021), how memorization changes as a function of training data size (Carlini et al., 2023; Wei et al., 2022), and how other memorized information can be detected based on model internal states (Haviv et al., 2023).

These works have collectively illustrated that there are some text snippets that LMs remember and can output verbatim or in closely paraphrased versions ("approximate memorization"; Ippolito